LOW-LEVEL DESIGN — CLASS 06

Effective Java Habits + Five Patterns

Static Factories · equals/hashCode · Flyweight · Facade · Observer · Command · Chain of Responsibility

---

Five new patterns today, each taught the same way: *a problem you'll recognize → the idea in one line → the simplest possible code → where it lives in real libraries → and exactly how it differs from the pattern you're about to confuse it with.*

Part 1 — Static Factory Methods (Effective Java, Item 1)

1.1 Three things a constructor cannot do

A constructor has no name. Quick — what does this mean?

```
Temperature t = new Temperature(30);      // 30 what? Celsius? Fahrenheit? Kelvin?
```

Worse: you *cannot* fix it with two constructors, because both would take a `double` — same signature, won't compile:

```
public Temperature(double celsius)     { ... }
public Temperature(double fahrenheit)  { ... }   // ✗ DOES NOT COMPILE — duplicate signature
```

Static factory methods have names, so the problem evaporates:

```
Temperature a = Temperature.fromCelsius(30);
Temperature b = Temperature.fromFahrenheit(86);
Temperature c = Temperature.fromKelvin(303.15);   // three "constructors", same parameter type
```

A constructor must allocate. `new` always creates a fresh object — no exceptions. A factory method is free to hand you an existing one:

```
Boolean.valueOf(true);      // NEVER allocates — returns the cached Boolean.TRUE
Integer.valueOf(100);       // returns a cached Integer (guaranteed for -128..127)
Integer.valueOf(100_000);   // allocates — outside the cache range
```

The caller's code is identical in all three lines. **The factory decides; the caller can't tell and shouldn't care.** That freedom is impossible with `new`.

A constructor must return exactly its own class. A factory can return any subtype — which lets an API hide its implementations entirely:

```
List<String> names = List.of("Rohit", "Amit");   // what class is this list, actually?
```

You don't know. It's some private JDK class (`ImmutableCollections.ListN`), chosen by the factory, possibly different for 0, 1, 2, or n elements. The JDK can swap it in any release and nobody's code breaks — because everyone holds the *interface*. This is DIP applied at the moment of creation, with the concrete class not merely quarantined but **invisible**.

1.2 You've been writing these all course

```
QuestionFilter.from(ctx)          // named, validates, parses — our Notes-02 DTOs
Cursor.encode(position)           // Notes 02
Caches.of("lru", 10_000)          // Notes 05 — hides the assembly recipe
Optional.empty()                  // JDK: returns the SAME shared instance every time
Duration.ofMinutes(5)             // named units — the Temperature trick
Executors.newFixedThreadPool(8)   // name tells you WHAT you're making
```

Rule of thumb from Effective Java: **consider static factories before public constructors.** Not always instead of — but consider.

1.3 Confusion #1 of the day: static factory ≠ Factory pattern

	Static factory method	Factory Method pattern (GoF)
What it is	an API technique: a named static method that creates	a *pattern*: creation deferred to a subclass/implementation override
Example	`List.of(...)`, `User.create(...)`	`Iterable.iterator()` — each collection builds *its own* iterator
When you say it	daily	mostly in interviews and old codebases

`User.of(...)` is not "using the Factory pattern." It's a well-named creation method. Say the right words in code review.

Part 2 — Four Effective Java Habits

Small habits, outsized effect on every design you'll ever do.

2.1 The `equals()` / `hashCode()` contract

The rule: if `a.equals(b)`, then `a.hashCode() == b.hashCode()`. Must. Always. No exceptions.

Why you care — the silent HashMap failure:

```
class UserId {                                      // equals overridden, hashCode FORGOTTEN
    private final String value;
    UserId(String value) { this.value = value; }

    @Override public boolean equals(Object o) {
        return o instanceof UserId u && u.value.equals(value);
    }
    // no hashCode() override → inherited from Object → based on memory address
}

Set<UserId> users = new HashSet<>();
users.add(new UserId("123"));
users.contains(new UserId("123"));     // FALSE. Equal objects. Set says no.
```

No exception, no warning — the set just quietly fails to find your object, because `HashSet` first jumps to a **bucket by hashCode** and only calls `equals` within that bucket. Two equal objects with different hash codes land in different buckets and never meet. Every hash-based structure (`HashMap`, `HashSet`, `distinct()`, `groupingBy`) breaks the same silent way.

The modern fix: records generate correct value-based `equals` + `hashCode` for free — one more reason our DTOs, filters, and cursors are records. Write them by hand only for non-record classes, and then *always as a pair* (`Objects.hash(...)` + `instanceof` pattern).

2.2 Defensive copying — and the *view vs snapshot* distinction

You know the attack from Notes 02:

```
List<String> members = new ArrayList<>(List.of("Rohit"));
Team team = new Team(members);       // Team stores the SAME list reference
members.add("Amit");                 // caller just mutated the Team from outside
```

The fix is `List.copyOf(members)` in the constructor. Today's refinement — two JDK tools that look identical and are not:

```
List<String> view = Collections.unmodifiableList(members);   // a WINDOW onto members
List<String> snap = List.copyOf(members);                    // a PHOTOGRAPH of members

members.add("Amit");
view.size();     // 2 — the view sees the change! (you can't write through it,
                 //      but the owner can still mutate underneath you)
snap.size();     // 1 — the snapshot is frozen forever
```

	`unmodifiableList`	`List.copyOf`
It is a	read-only **view** over the original	independent unmodifiable **copy**
Original mutates →	view changes too	copy unaffected
Use when	exposing live internal state read-only	protecting an object's state at a boundary

For constructors of immutable objects: **always the snapshot.** A view would leave a mutation tunnel open.

2.3 Try-with-resources — lifecycle as a contract

The old ritual every Java developer got wrong at least once:

```
InputStream in = new FileInputStream("data.txt");
try {
    // use in ... (and if THIS throws, and close() also throws, the first exception is lost)
} finally {
    in.close();                    // forgot this once → leaked file handle
}
```

The modern form:

```
try (InputStream in = new FileInputStream("data.txt")) {
    // use in
}                                  // close() called automatically — even on exception,
                                   // even for multiple resources, in reverse order
```

The design lesson is bigger than the syntax: the JDK didn't fix this with documentation ("please remember to close!") — it fixed it with an **interface**: anything implementing `AutoCloseable` plugs into the language's cleanup machinery. When you write a class that holds a connection, file, or lock, implement `AutoCloseable` and your users get correctness *by structure, not by memory*. That's the recurring theme of this course wearing a new coat: make the right thing automatic.

2.4 One-line reminder: records are not *deeply* immutable

```
record Order(List<Item> items) { }        // the FIELD is final; the LIST is not
```

A record freezes its references, not the objects behind them — `List.copyOf` in the compact constructor remains your job (Notes 02). Records + snapshots + correct equals/hashCode: that's the complete value-object recipe.

Part 3 — Flyweight

3.1 The problem — 10 million trees

A game renders a forest of 10 million trees. Each tree object holds:

```
class Tree {
    int x, y, size;                    // different for every tree
    String name;                       // "Oak" — 8 million times
    byte[] texture;                    // oak.png, ~50 KB — 8 million times?!
    Color color;                       // brown — 8 million times
}
```

10M trees × 50 KB of texture each ≈ **500 GB** for one forest. Game over — literally — before it starts. But look at the fields again: only `x, y, size` actually differ per tree. The rest is identical across all 8 million oaks.

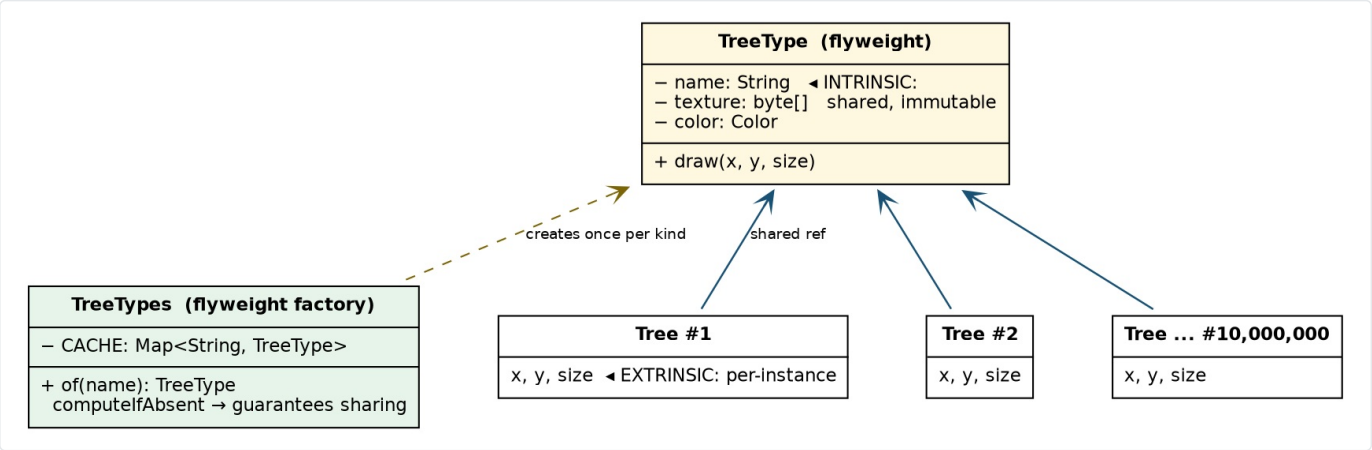
3.2 The idea, one line

> **Split the object's state in two: share the common part (one object per *kind*), keep only the varying part per instance.**

The GoF names: **intrinsic** state = shared, belongs to the kind (name, texture, color); **extrinsic** state = per-instance (x, y, size).

```java
// the SHARED part — one instance per tree KIND, ever
record TreeType(String name, byte[] texture, Color color) {
    void draw(int x, int y, int size) { /* render texture at x,y scaled to size */ }
}

// the factory guarantees the sharing (computeIfAbsent = get-or-create)
class TreeTypes {
    private static final Map<String, TreeType> CACHE = new HashMap<>();
    static TreeType of(String name) {
        return CACHE.computeIfAbsent(name, n -> load(n));   // "Oak" always → the SAME object
    }
}

// each tree is now three ints and one shared reference
record Tree(int x, int y, int size, TreeType type) {
    void draw() { type.draw(x, y, size); }
}
```

10M trees now cost: 10M × (3 ints + 1 reference) + **3 TreeType objects total**. Megabytes, not terabytes.



Flyweight — millions of Trees carry only extrinsic state (x, y, size) and share one immutable TreeType per kind, guaranteed by the factory.

Two things to notice:

1. **The factory is what makes it a flyweight.** Without `TreeTypes.of` forcing reuse, callers would `new TreeType(...)` per tree and nothing is shared. (Static factory method → enabling flyweight: Part 1 paying rent already.)
2. **Flyweights must be immutable.** 8 million trees point at ONE oak `TreeType` — if anyone could mutate it, changing "my" tree's color would repaint 8 million trees. Sharing is only safe because the shared thing cannot change. (Part 2 paying rent.)

3.3 You use flyweights every day without knowing

The Java String pool:

```java
String a = "hello";
String b = "hello";
a == b;                    // true — SAME object. The JVM pools string literals.
```

Every `"hello"` literal in your entire program is one shared object. That's a flyweight factory built into the language.

The Integer cache (from Part 1):

```java
Integer.valueOf(100) == Integer.valueOf(100);       // true  — cached (-128..127)
Integer.valueOf(1000) == Integer.valueOf(1000);     // false — outside the cache
```

(Which is also exactly why you must compare objects with `equals`, never `==` — the famous interview trap is just the flyweight cache boundary.)

Text editors: a 10-million-character document doesn't hold 10 million glyph objects — every letter 'a' shares one 'a' glyph (intrinsic: shape, font metrics); position in the document is extrinsic. This was the GoF's original motivating example.

3.4 Confusions

Flyweight vs cache. Every flyweight uses a cache; not every cache is a flyweight. A cache's purpose is usually *avoiding recomputation or I/O* (our Notes-05 cache: DB results, any values, evictable, mutable-ish). A flyweight's purpose is

deduplicating identical immutable state in memory — nothing is "recomputed," the entries are never evicted mid-use, and sharing is the point, not speed.

Flyweight vs Singleton.

	Singleton	Flyweight
How many instances	exactly **one per class**	**one per distinct value** — as many as there are kinds
The question it answers	"there must only ever be one Config"	"must 8 million oaks each carry the oak texture?"
Identity	of the class	of the value (Oak vs Pine vs Mango)

Memory trick: Singleton = *one*. Flyweight = *one per kind, shared by many*.

Part 4 — Facade

4.1 The problem — what's behind the "Buy Now" button?

You tap **Buy Now** on Amazon. One tap. Behind it:

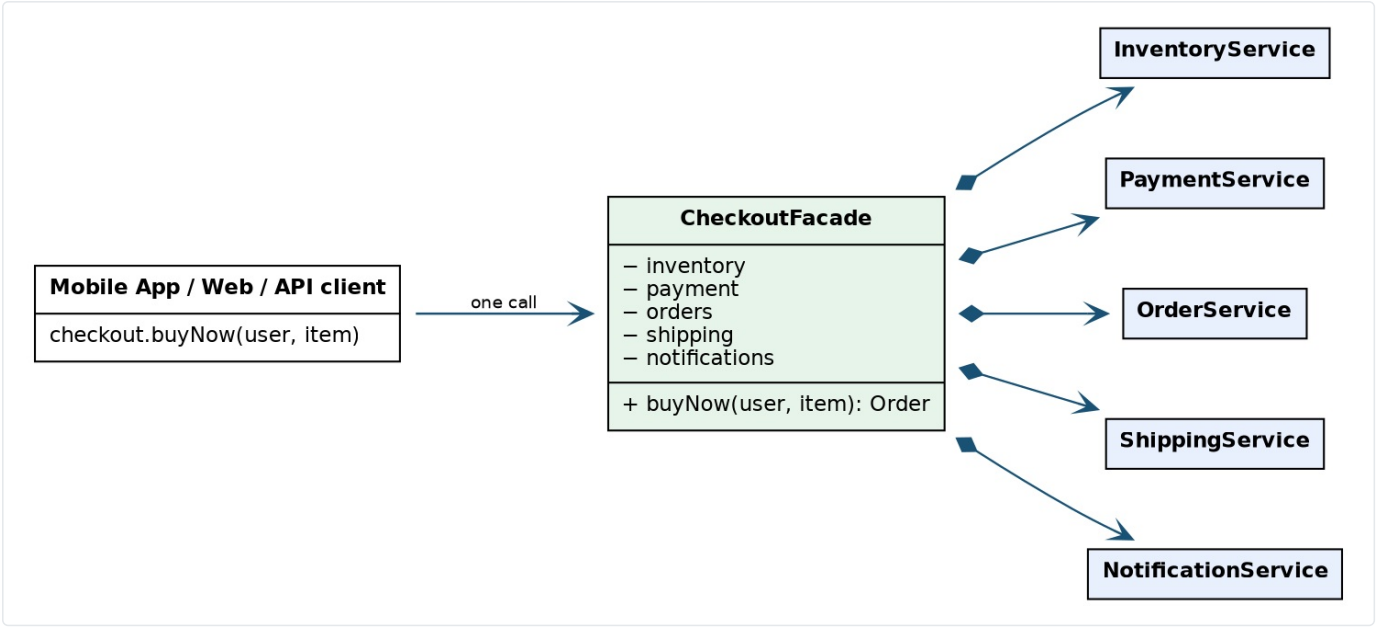
```
inventory.reserve(item);
payment.charge(user, amount);
Order order = orders.create(user, item);
shipping.createShipment(order);
invoice.generate(order);
notifications.send(user, "Order placed!");
```

Should the mobile app's button-click handler know these six subsystems, their order, and their failure handling? Should the website duplicate the same six calls? When step 7 (loyalty points) is added, do we update every client?

4.2 The idea, one line

> **Put one simple, intention-named entry point in front of a complicated subsystem.**

```
public class CheckoutFacade {

    private final InventoryService inventory;      // injected, as always
    private final PaymentService payment;
    private final OrderService orders;
    private final ShippingService shipping;
    private final NotificationService notifications;

    public Order buyNow(User user, Item item) {    // the button IS this method
        inventory.reserve(item);
        payment.charge(user, item.price());
        Order order = orders.create(user, item);
        shipping.createShipment(order);
        notifications.send(user, "Order placed!");
        return order;
    }
}
```



Facade — one intention-named entry point in front of the subsystem; clients know one verb, the subsystems remain unchanged behind it.

Clients now know one verb: `checkout.buyNow(user, item)`. The subsystems still exist, unchanged, and power users may still call them directly — a facade **simplifies access, it doesn't seal the room.** New step? One place changes; mobile, web, and API clients never notice.

(Sound familiar? Our `QuestionService` composing repositories in Notes 02 already had facade energy — a service layer often IS a facade over lower layers. The pattern name just makes the intent sayable.)

4.3 Real facades you already call

`java.nio.file.Files`:

```
String text = Files.readString(path);
```

One line. Underneath: filesystem providers, channels, byte buffers, charset decoding, resource cleanup. `Files` is a facade of static methods over the whole NIO machinery — and note, *a facade needn't be a fancy object with an interface*; a well-named class of entry points qualifies.

Spring's `JdbcTemplate`: one `query(sql, mapper)` call hides connection acquisition, statement prep, result iteration, exception translation, and guaranteed cleanup — the try/finally jungle every pre-Spring codebase repeated a thousand times.

4.4 Confusion: Facade vs Adapter

They're both "a class in front of another thing," so students merge them. The difference is **why** the class exists:

	Adapter	Facade
The complaint	"the interface is **incompatible** — wrong shape"	"the interface is **complicated** — too many pieces"
What it does	converts one interface into the one the client requires	condenses many calls into one intention-level call
Typically wraps	**one** object	a whole **subsystem**
Our example	`StripeAdapter implements PaymentGateway` (Notes 01)	`CheckoutFacade`, `Files`

> **Adapter changes the shape. Facade shrinks the surface.**

If you're translating method names and parameter types → adapter. If you're bundling a sequence of calls behind one verb → facade.

Part 5 — Observer

5.1 The problem — who needs to know the price changed?

A stock's price ticks. Interested parties: the mobile app, the alerts engine, the analytics pipeline, the trading dashboard. First instinct:

```
class Stock {
    private MobileApp app;                // Stock now imports the UI?!
    private AlertService alerts;          // and the alerting system?!
    private Analytics analytics;          // and the data pipeline?!

    void setPrice(double p) {
        app.refresh(p); alerts.check(p); analytics.record(p);
    }
}
```

`Stock` — a domain object — now depends on every consumer in the company. Add a fifth consumer, edit `Stock`. Draw the arrows: they all point the wrong way (a Notes-01 alarm should be ringing).

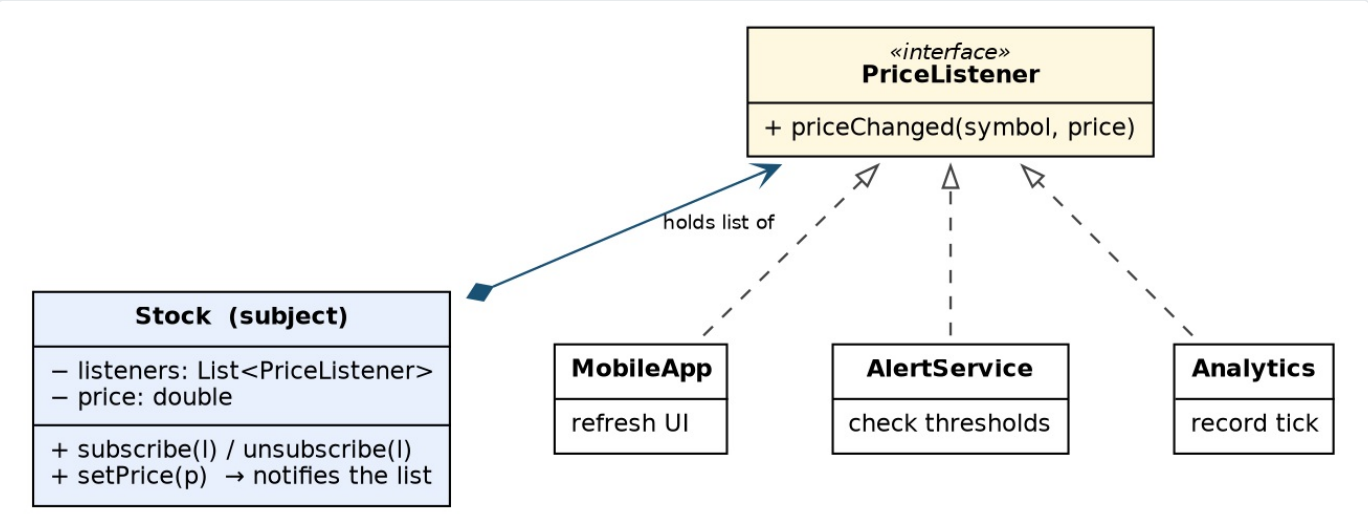
The everyday version of this pattern: you don't call YouTube every hour asking "did they upload?" You **subscribe** once; the channel notifies everyone on its list when something happens. The channel doesn't know you — it knows *a list*.

5.2 The idea, one line

> **The subject keeps a list of subscribers behind an interface, and notifies the list when its state changes — knowing none of them concretely.**

```
interface PriceListener {                          // the only thing Stock knows
    void priceChanged(String symbol, double price);
}

class Stock {
    private final List<PriceListener> listeners = new ArrayList<>();
    private double price;

    void subscribe(PriceListener l)  { listeners.add(l); }
    void unsubscribe(PriceListener l){ listeners.remove(l); }

    void setPrice(double newPrice) {
        this.price = newPrice;
        for (PriceListener l : listeners)          // notify — whoever they are
            l.priceChanged("GOOG", newPrice);
    }
}
```

```
stock.subscribe((sym, p) -> mobileApp.refresh(p));     // a lambda is a listener —
stock.subscribe((sym, p) -> alerts.check(sym, p));     // functional interface, Notes 04
stock.subscribe((sym, p) -> analytics.record(sym, p));
```



Observer — the subject holds a list of the interface; concrete subscribers implement it. Stock depends on nobody concrete.

The dependency arrows flipped: consumers depend on `Stock`'s listener interface; `Stock` depends on nobody. Adding the fifth consumer = one `subscribe` call at the composition root. (Also note: `Stock` calls code it has never heard of — observer is Inversion of Control between peers, "don't call us, we'll call you" again.)

5.3 You've written observers for years

- **Every UI event listener.** `button.addEventListener("click", handler)` in JavaScript, `button.setOnClickListener(...)` on Android — the button is the subject, your handler subscribes. If you've written a click handler, you've used Observer.
- `PropertyChangeSupport` (JavaBeans): the JDK's boxed implementation — `addPropertyChangeListener(...)`, `firePropertyChange("price", old, now)` — it maintains the list and dispatches so you don't hand-roll it.
- `java.util.concurrent.Flow` (Publisher/Subscriber): Observer grown up for production streams — it adds **backpressure** (`subscription.request(n)`: the subscriber controls the flow rate so a fast publisher can't drown it), async delivery, and error/completion signals. Teach it as: *Observer is the seed; reactive streams are the tree.*
- **In our Stack Overflow system:** "notify followers when a question gets an answer" — the followers list is observers; you'll wire it in the final project.

5.4 Confusion: Observer vs Pub/Sub

	Observer	Publish/Subscribe
Who holds the subscriber list	**the subject itself** (`Stock.listeners`)	a **broker in the middle** (Kafka, RabbitMQ, an EventBus)
Publisher and subscriber	reference each other's interfaces, same process	don't know each other AT ALL — only the broker/topic
Coupling	loose	looser (can be different processes, languages, uptimes)
Delivery	synchronous method call (usually)	usually async, durable, retried

Memory trick: **Observer = the channel keeps its own subscriber list. Pub/Sub = a post office sits in between.** Same intent, one extra hop — and that hop is what lets systems scale across machines.

One production honesty note: our simple loop notifies synchronously, in subscription order, on the caller's thread — one slow or throwing listener stalls or breaks the others. Real implementations copy the list, isolate exceptions, and often dispatch async. Mention it in interviews; it's the difference between the diagram and the system.

Part 6 — Command

6.1 The problem — the restaurant order slip

Watch what happens when you order food. You tell the waiter; the waiter **writes it on a slip** and clips it to the kitchen rail. That slip is a small miracle of design:

- It **packages the whole request**: table 7, two masala dosas, extra chutney.
- It can be **queued** (the rail), **reordered** (VIP first), **handed to any cook**, **cancelled** (pull it off), **logged** (your bill is the slip history), and executed **later** — long after you finished speaking.

If the waiter instead *shouted your order directly at a specific cook*, none of that would be possible. The slip decouples *requesting* from *executing*.

6.2 The idea, one line

> **Turn a request into an object — so it can be stored, queued, logged, undone, and executed by someone who doesn't know what it does.**

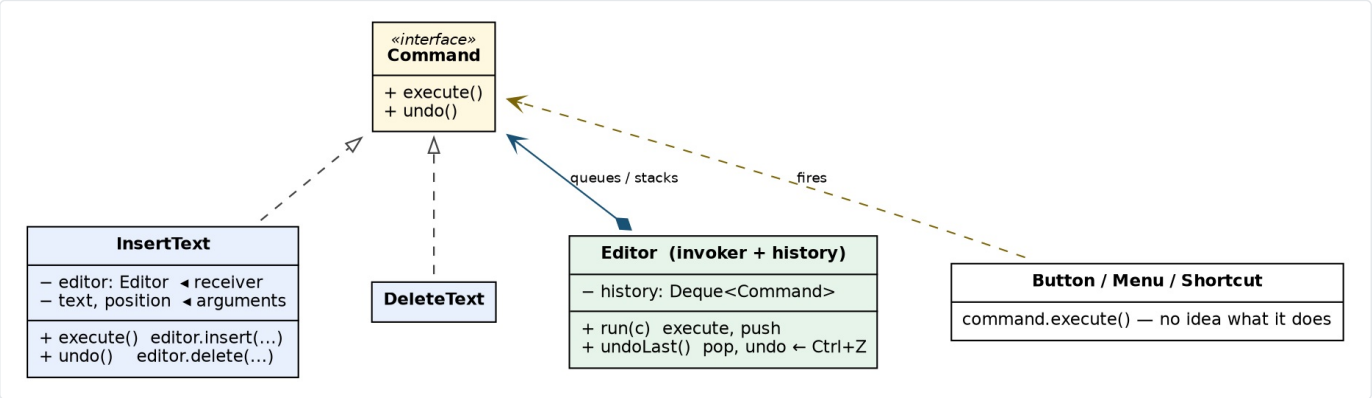
The editor example — because it leads straight to the killer feature, **undo**:

```java
interface Command {
    void execute();
    void undo();
}

class InsertText implements Command {
    private final Editor editor;         // the RECEIVER
    private final String text;           // the ARGUMENTS
    private final int position;          // — the whole request, packaged

    InsertText(Editor editor, String text, int position) {
        this.editor = editor; this.text = text; this.position = position;
    }
    public void execute() { editor.insert(position, text); }
    public void undo()    { editor.delete(position, text.length()); }   // knows its own reverse
}
```

```java
class Editor {                           // ...and undo falls out of the design:
    private final Deque<Command> history = new ArrayDeque<>();

    void run(Command c) {
        c.execute();
        history.push(c);                 // every executed command remembered
    }
    void undoLast() {
        if (!history.isEmpty()) history.pop().undo();     // Ctrl+Z is a stack pop
    }
}
```

Ctrl+Z was always mysterious until you see it's just: *operations are objects, executed objects go on a stack, undo pops.* And the invoker is fully generic — the same button/menu-item/shortcut code runs *any* command:

```java
toolbar.onClick(copyCommand);
menu.onSelect(copyCommand);              // three UI elements, one operation object,
shortcut.bind("Ctrl+C", copyCommand);    // zero knowledge of what "copy" means
```



Command — the request is an object carrying receiver + arguments + its own undo; invokers fire it, the history stack makes Ctrl+Z a pop.

6.3 You've been queuing commands all along

Every `Runnable` you've ever submitted:

```java
executor.submit(() -> processOrder(order));    // a Runnable IS a command:
                                               // request packaged as an object,
                                               // queued, executed later, by a thread
                                               // that has NO idea what it's running
```

`ExecutorService` is a command invoker; its work queue is the kitchen rail. So is every job queue, every `@Scheduled` task, every message a worker consumes — **a task queue is the Command pattern at industrial scale.** Swing's `Action` is the classic GUI version (one action object shared by button + menu + shortcut, with shared enabled/disabled state).

Notice the Notes-04 pattern repeating: for fire-and-forget commands, **a lambda/`Runnable` is the lightweight form** — the language absorbed the simple case. The full class earns its keep exactly when the command needs **state**: undo information, retry count, an id for logging. Same rule as Strategy: *stateless → lambda; stateful → class.*

6.4 Confusion: Command vs Strategy

They look identical — one interface, one method:

```
interface Strategy { int calc(Trip t); }
interface Command  { void execute(); }
```

The difference is **intent and lifetime**:

	Strategy	Command
Represents	**HOW** to do something — one of several interchangeable algorithms	**WHAT** to do — one specific request, captured
Carries	just logic (usually stateless)	receiver + arguments + possibly undo/state
Cardinality	a few, chosen from, long-lived	created per request, possibly thousands, often short-lived
Typical home	injected via constructor, used repeatedly	pushed into queues/stacks/logs
Our examples	fare policy, eviction policy, `SortBy`	`InsertText`, a submitted `Runnable`, an order slip

Memory trick: **Strategy is a setting. Command is a request.** You *configure* a strategy; you *fire* a command.

Part 7 — Chain of Responsibility

7.1 The problem — every request must run the gauntlet

Every HTTP request into our Stack Overflow API should pass: authentication → rate limiting → validation → then the controller. The nested-if version:

```
if (isAuthenticated(req)) {
    if (rateLimitOk(req)) {
        if (isValid(req)) {
            controller.handle(req);
        } else { reject(400); }
    } else { reject(429); }
} else { reject(401); }
```

Pyramid of doom — and it's *one* pyramid per endpoint, the checks can't be reordered per route, can't be tested alone, and adding a logging step means editing every pyramid.

The everyday version: **customer support escalation.** You call support; L1 either solves it or escalates to L2; L2 solves or escalates to L3; L3 to the manager. Each level knows only two things: *can I handle this?* and *who's next?* Nobody holds the whole pyramid.

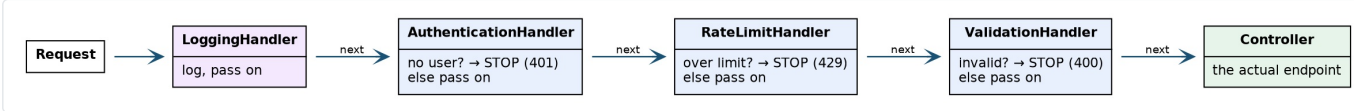
7.2 The idea, one line

> **Line up handlers; each one either handles the request, stops it, or passes it to the next.**

```
interface Handler {
    void handle(Request request, Handler next);
}

class AuthenticationHandler implements Handler {
    public void handle(Request req, Handler next) {
        if (req.user() == null) throw new UnauthorizedException();   // STOP the chain
        next.handle(req, ...);                                        // or pass it on
    }
}

class RateLimitHandler implements Handler {
    private final RateLimiter limiter;              // remember Problem 3, Set 2?
    public void handle(Request req, Handler next) {
        if (!limiter.allow(req.user())) throw new TooManyRequestsException();
        next.handle(req, ...);
    }
}
```

A tidier modern shape — the chain as a *list* rather than hand-wired `setNext` links:

```java
class Pipeline {
    private final List<Handler> handlers;           // order = configuration, in ONE place
    private final Controller controller;

    void process(Request req) { run(req, 0); }
    private void run(Request req, int i) {
        if (i == handlers.size()) { controller.handle(req); return; }
        handlers.get(i).handle(req, r -> run(r, i + 1));      // "next" is just a callback
    }
}

// composition root: the whole gauntlet, declared as data
var pipeline = new Pipeline(List.of(
        new LoggingHandler(),
        new AuthenticationHandler(),
        new RateLimitHandler(limiter),
        new ValidationHandler()
), controller);
```



Chain of Responsibility — the request runs the gauntlet; each handler passes it on or stops it before the controller.

The wins: each check is one small class, testable alone; the order is data in one place; per-route chains just build different lists; adding a step touches nothing but the list.

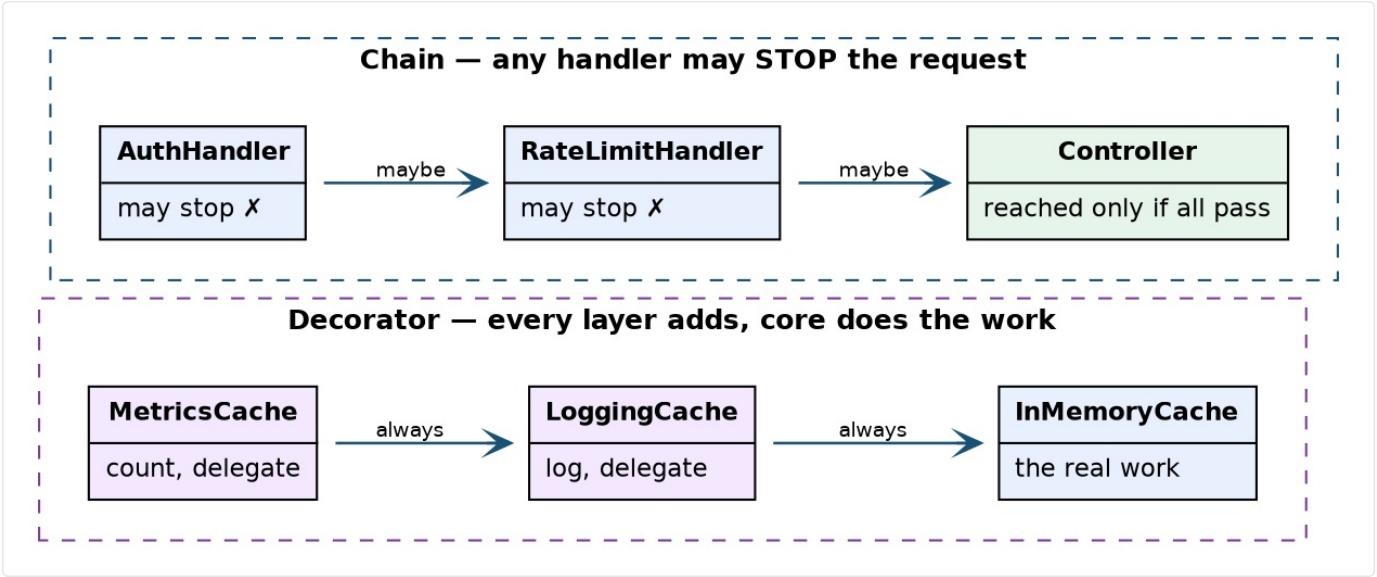
7.3 You're standing inside this pattern right now

- **Javalin — our own web layer:** `app.before(...)` handlers run in order before your endpoint, and any of them can halt the request. Our Notes-02 API already had an empty chain waiting for us.
- **Servlet `FilterChain`** — the grandfather: each filter does its work and either calls `chain.doFilter(req, res)` ("I'm done, next!") or doesn't (chain stopped). Spring Security is famously "just" a long filter chain.
- **Express/Node middleware** — `app.use((req, res, next) => ...)`: the `next()` call *is* the chain, in a language half of you use daily.
- **Java exception handling itself:** a thrown exception travels **up the call stack** until some frame has a matching `catch` — each frame either handles it or passes it up. Chain of Responsibility is built into the language semantics you've used since day one.

7.4 Confusion: Chain vs Decorator (the structural twins)

Both are objects linked to objects of the same interface, each calling the next — the *diagrams look identical*. The intent differs:

	Decorator	Chain of Responsibility
Each link's job	**add behavior**, then (almost) always delegate	**decide**: handle / stop / pass along
Does the call reach the end?	essentially always — the core object does the real work	maybe not — any handler may short-circuit
Built around	an object being **enriched** (the cache, the stream)	a request being **routed/vetted**
Removal test	remove a decorator → same behavior, minus one feature	remove a handler → a whole check disappears (security hole!)
Ours	`LoggingCache(new InMemoryCache(...))`	auth → rate-limit → validate → controller



The structural twins — identical diagrams, different intent: decorators always delegate to the working core; chain handlers may short-circuit.

Memory trick: **Decorator wraps a thing. Chain screens a request.** If every layer runs and the middle does the real work → decorator. If any layer can say "denied" → chain.

(Bonus connection: put `Command` objects in the chain's list and you get a composable processing pipeline — Apache Commons Chain is literally this, a chain of Commands. Patterns compose; that's next class's whole theme.)

Part 8 — The Mental Model (print this)

One question per pattern — if you can ask the question, you can pick the pattern:

Pattern	The question it answers
Static factory	"Can creation be named, cached, or hidden behind the interface?"
Flyweight	"Are millions of objects carrying identical immutable state?"
Facade	"Can six calls into a subsystem become one intention-named call?"
Observer	"How do N parties react to a change without the subject knowing them?"
Command	"Should this *request* be an object — queued, logged, undone?"
Chain of Responsibility	"Should this request pass a sequence of checks, any of which may stop it?"

The confusion map — the six distinctions that matter

```
Static factory vs Factory pattern → technique with a name  vs  creation via subclass override
Flyweight     vs Singleton        → one per VALUE, shared   vs  one per CLASS
Flyweight     vs Cache            → dedup immutable state   vs  avoid recomputation
Facade        vs Adapter          → shrink the surface      vs  change the shape
Observer      vs Pub/Sub          → subject holds the list  vs  broker in the middle
Command       vs Strategy         → a request (WHAT)        vs  an algorithm (HOW)
Chain         vs Decorator        → may stop the request    vs  always enriches and delegates
```

Part 9 — Closing Exercise (10 minutes, whiteboard)

Design the request-processing layer of our Stack Overflow API, with these requirements: authenticate → rate-limit → validate → run the endpoint; log every request; when a request completes, metrics, audit, and (for writes) follower-notification must all react; the mobile team wants one simple `process(request)` entry point; and slow work (sending notifications) must not block the response.

Expected mapping — say it in pattern vocabulary:

```
one entry point            →  Facade            RequestProcessor.process(req)
auth / rate-limit / valid  →  Chain             ordered handlers, any may stop
metrics + audit + notify   →  Observer          RequestCompleted event, subscribers react
slow follower-notification →  Command           packaged as a task, queued to an executor
handler & task creation    →  Static factories  named, wired at the composition root
```

And the closing lesson, which is the bridge to everything after this class:

> **No real system uses one pattern.** Good design is small patterns composed, each owning one kind of change — creation, notification, screening, deferral. You now have the vocabulary; from here on we use it in full-system designs.